SOL PLAATJE
UNIVERSITY

Restaurant System Documentation (Android & Desktop)

Module Information

- **Module Name**: Mobile App Development/Programming II
- **APP Name:** Urban Style Innovators
- **By: Menzi Sigwebela**

Project Overview

Taste Heaven Restaurant System is a comprehensive restaurant management solution deployed as both an Android mobile application and a Java-based desktop application. The Android app leverages Firebase for real-time data management, while the desktop app uses Java Swing (JFrame) for the GUI and MySQL for persistent storage. Both platforms offer identical functionality, catering to Customers, Waiters, Chefs, and Managers, with features like menu browsing, order placement, table reservations, and inventory management.

1. **Problem Statement**

Taste Heaven Restaurant currently faces operational inefficiencies due to its reliance on manual processes for order management, table reservations, and inventory tracking. These issues lead to delayed service, reservation conflicts, and limited real-time oversight, ultimately impacting customer satisfaction and business performance. The absence of an integrated, digital solution hampers communication between staff roles such as waiters, chefs, and managers, and creates friction in customer interactions. There is a critical need for a unified, cross-platform restaurant management system that streamlines operations, enhances user experience, and provides real-time data synchronization across Android and desktop platforms. The system must address the unique functional needs of each stakeholder—Customers, Waiters, Chefs, and Managers—while maintaining high standards of performance, scalability, security, and usability.

2. **Solutions**

- **Authentication & Security:**
    - Secure login systems using Firebase Authentication (Android) and hashed passwords (Desktop).
    - Encrypted communication to protect user data and transaction details.
- **Performance Optimization:**
    - Menu and order data load within 2 seconds using asynchronous Firebase queries and optimized MySQL queries.
- **Scalability:**
    - Firebase handles real-time data for up to 1,000+ users.
    - Desktop app supports large-scale data processing through optimized MySQL database schema.

- **Usability & Accessibility:**
    - Responsive designs tailored to mobile touch inputs and desktop mouse/keyboard interaction.
    - Consistent user experience and design language across platforms for ease of training and user onboarding.

3. Requirements Analysis (Planning Phase)

Objective: Clearly identify and document user needs and system requirements for both Android and desktop applications.

Stakeholder Analysis

Customers:

- A visually appealing, user-friendly interface to:
    - Browse categorized menus (breakfast, lunch) with images, prices, and descriptions.
    - Add items to a cart and submit orders.
    - Make table reservations based on live availability, date/time, and guest count.
    - Track real-time order statuses and submit feedback directly through the app.

Waiters:

- Dashboard to:
    - View customer orders by table, including status updates and special requests.
    - Update order statuses in real-time (e.g., "Accepted", "Ready", "Delivered").
    - Manage table assignments efficiently to avoid service overlaps or delays.

Chefs:

- Simple yet effective interface to:
    - View a queue of accepted orders with item details and table numbers.

- Mark orders as “Ready” to notify both waiters and customers.

Managers:

- Comprehensive control panel to:
  - Add, edit, or remove menu items.
  - Monitor and manage inventory levels with stock alerts and reorder thresholds.
  - Review, approve, or reject table reservations.
  - Access live statistics on revenue, active users, and overall system usage.

3.1. Functional Requirements

Output Requirements	• The system must generate a daily sales report showing items sold, quantity, and total income, grouped by category. • The manager dashboard must display a real-time summary of total reservations, pending orders, and inventory alerts. • The system must email chefs a summary of all pending orders every 30 minutes. • The inventory module must produce a weekly stock reorder report sorted by lowest stock levels.
Input Requirements	• Customers must enter reservation details including date, time, number of guests, and contact information. • Waiters must scan or enter table numbers before assigning new orders. • The manager must input new menu items with name, description, price, category, and image. • Users must enter login credentials (email and password) with secure form validation.
Process Requirements	• The Android app must sync new reservations to Firebase in real time and update availability across devices. • The desktop system must check stock availability before allowing new orders. • The system must automatically calculate total bill amounts, including VAT. • When a customer submits feedback, the system must store it with a timestamp and user ID.

Performance Requirements	• The Android app must load menu items in under 2 seconds over a 4G connection. • The desktop app must support at least 50 simultaneous logins with no performance degradation. • New orders must be displayed on the chef dashboard within 5 seconds of submission. • Order status updates must reflect across all platforms instantly (Firebase push sync).
Control Requiremnts	• Only managers can approve or reject reservations and edit menu items. • Users must be authenticated before accessing their respective dashboards. • All transactions must be logged with timestamps for audit purposes. • The system must restrict chef dashboard access to users with the 'Chef' role only.

3.2 Non-Functional Requirements

To ensure the system performs efficiently, securely, and remains user-friendly, the following non-functional requirements have been identified:

3.2.1. Performance

- The system must load menu items within **2 seconds** on both mobile and desktop platforms.
- Table reservations and order submissions should be processed in **under 3 seconds**.
- Reporting features on the desktop app should generate within **5 seconds** upon request.

3.2.2. Scalability

- The system should support up to **100 concurrent users**, especially during peak hours (e.g., lunch/dinner rush).
- The backend database must handle a minimum of **10,000 data records** (e.g., orders, reservations) without a noticeable drop in performance.

3.2.3. Availability

- The application must maintain **99.5% uptime** during operational hours.
- The Android app should offer limited offline functionality, with synchronization occurring once internet connectivity is restored.

3.2.4. Security

- All sensitive user data must be transmitted over **secure HTTPS/TLS connections**.
- Role-based access control (RBAC) must be implemented, supporting roles such as **Customer, Waiter, Chef, and Manager**.
- User credentials should be **hashed and securely stored** using industry-standard methods (e.g., Firebase Authentication or BCrypt).
- Access to customer data and orders must be restricted based on authenticated user roles.

3.2.5. Usability

- Interfaces must be **intuitive, accessible, and user-friendly**, catering to both technical and non-technical users.
- UI elements like fonts, icons, and buttons must follow accessibility guidelines, ensuring ease of use for people with disabilities.
- Error and confirmation messages must be **clear, concise, and non-technical** to support smooth user interactions.

3.2.6. Portability

- The Android application must be compatible with devices running **Android 10 (API level 29)** and above.
- The desktop application should run seamlessly on major operating systems, including **Windows, macOS, and Linux**, provided Java 8 or higher is installed.

3.2.7. Maintainability

- The system's codebase should be **modular, reusable, and well-documented** to allow for easy maintenance and future enhancements.
- The database schema must follow proper normalization (ideally up to **3NF or 4NF**) to ensure data consistency and scalability.

3.2.8. Reliability

- The system should recover gracefully from common failures, such as a database connection timeout or network interruption.
- Firebase features must include retry logic for handling temporary network issues on the mobile app.

3.2.9. Localization (Future-Oriented)

- The system should be designed to allow future integration of **multiple languages** for a diverse user base.

3.2.10. Compliance

- All user data handling must comply with **POPIA (Protection of Personal Information Act)** and other relevant data protection laws in South Africa.

4. System Design (Design Phase)

4.1 Context Diagram (Project Scope)

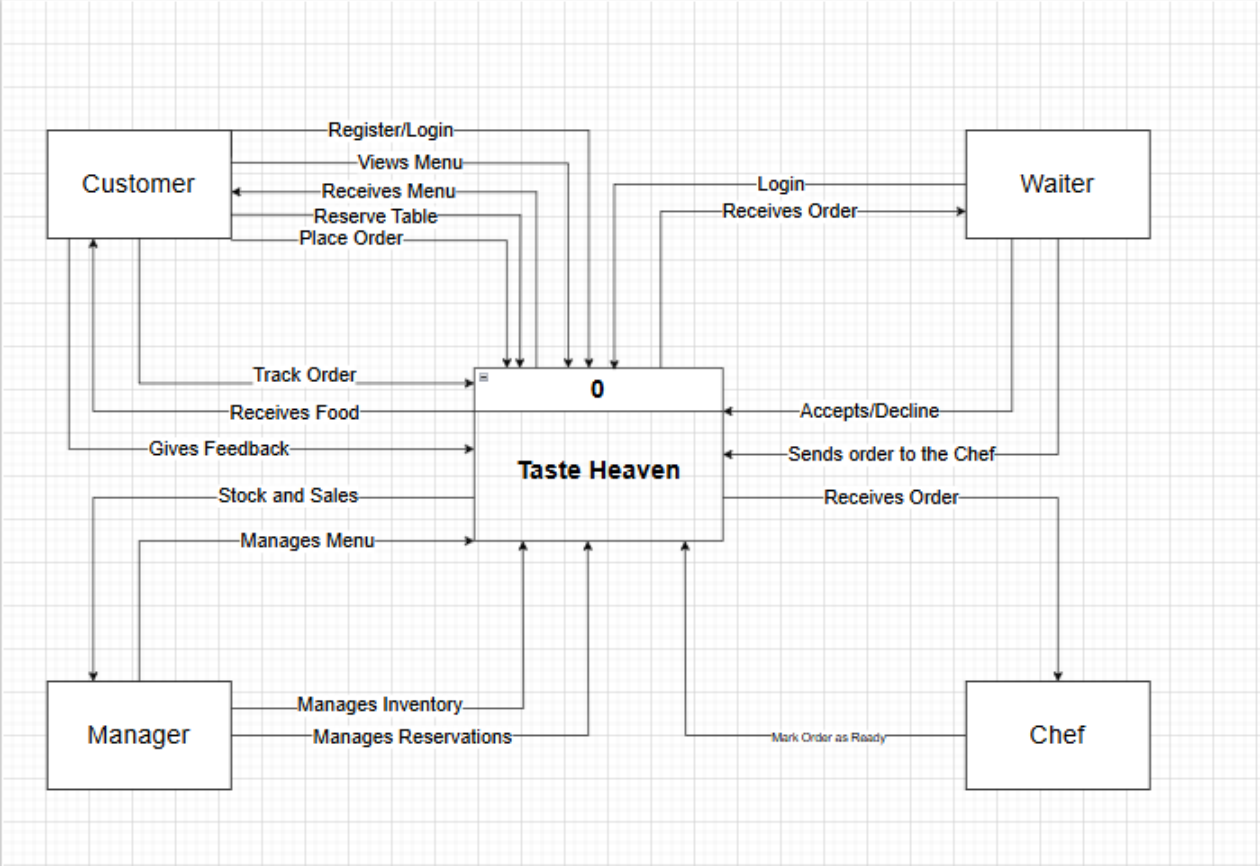
Context Diagram

[Customer] --> (Place Order, Reserve Table, Track Order, Provide Feedback) -->

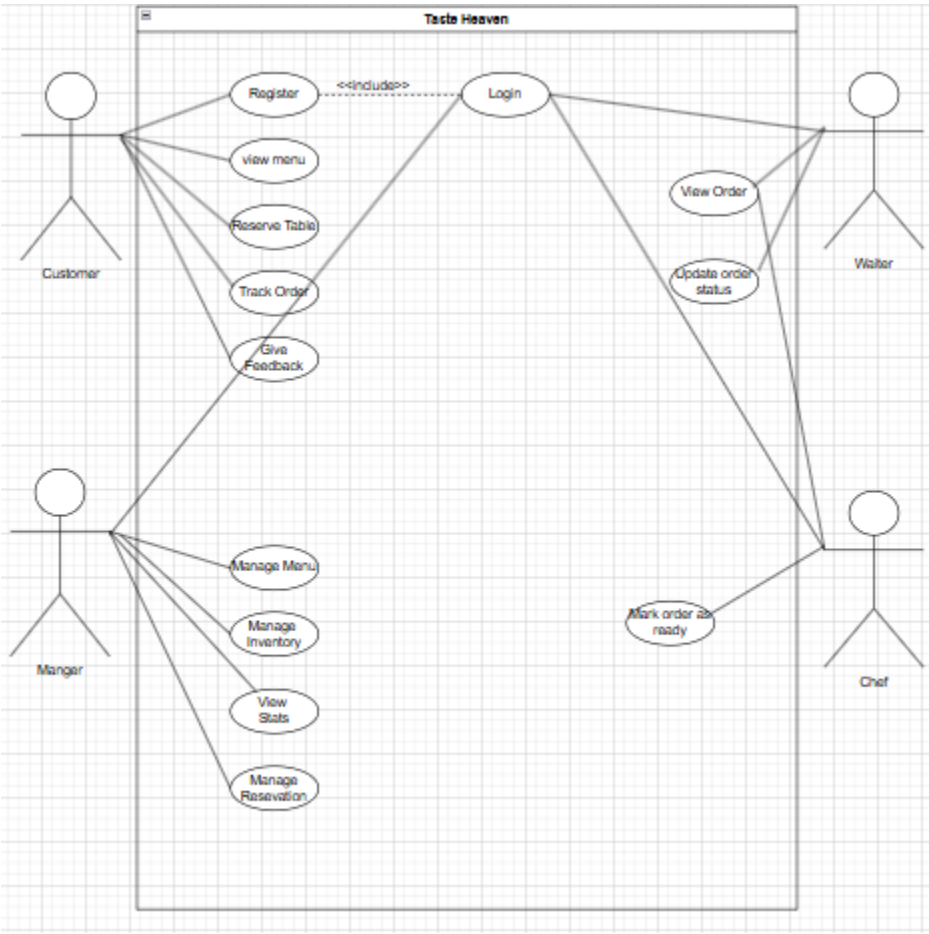
[Waiter] --> (Manage Orders, Update Status) --> [Taste Heaven Restaurant System]

[Chef] --> (View Orders, Mark Ready) --> [Taste Heaven Restaurant System]

[Manager] --> (Manage Menu, Inventory, Reservations, View Stats) →



4.2 Use Case Diagram (Functional interaction)



The Use Case Diagram captures functional interactions for both platforms:

- **Actors**: Customer, Waiter, Chef, Manager
- **Use Cases**:
    - Customer: Register, Sign in, View Menu, Place Order, Reserve Table, Track Order, Provide Feedback
    - Waiter: View Orders, Update Order Status
    - Chef: View Orders, Mark Order as Ready
    - Manager: Manage Menu, Manage Inventory, Approve/Reject Reservations, View Statistics

4.3 Normalization

Unnormalized Form (UNF)

OrderID	CustomerName	Contact	ItemsOrdered	Quantities	WaiterName	TableNumber
001	Karabo	0781234, 0722…	Burger, Coke, Fries	1, 1, 2	Neo	12
002	Ayanda	0655678	Pasta, Juice	1, 2	Neo	5

Issues:

- ItemsOrdered & Quantities are multi-valued.
- Contact can itself be multi-valued.
- Repeating groups → not atomic.

First Normal Form (1NF)

All fields are atomic; each item and quantity gets its own row.

OrderID	CustomerName	Contact	Item	Quantity	WaiterName	TableNumber
001	Karabo	0781234	Burger	1	Neo	12
001	Karabo	0781234	Coke	1	Neo	12
001	Karabo	0781234	Fries	2	Neo	12
002	Ayanda	0655678	Pasta	1	Neo	5
002	Ayanda	0655678	Juice	2	Neo	5

Second Normal Form (2NF)

Remove partial dependencies by splitting off the order line details.

Orders

OrderID	CustomerName	Contact	WaiterName	TableNumber
001	Karabo	0781234	Neo	12
002	Ayanda	0655678	Neo	5

OrderDetails

OrderID	Item	Quantity
001	Burger	1
001	Coke	1
001	Fries	2
002	Pasta	1
002	Juice	2

Third Normal Form (3NF)

Remove transitive dependencies by isolating the waiter–table assignment.

Orders

OrderID	CustomerName	Contact	TableNumber
001	Karabo	0781234	12
002	Ayanda	0655678	5

OrderDetails

OrderID	Item	Quantity
001	Burger	1
001	Coke	1
001	Fries	2
002	Pasta	1
002	Juice	2

TableAssignments

TableNumber	WaiterName
12	Neo
5	Neo

Fourth Normal Form (4NF)

Eliminate any remaining multi-valued dependencies. Here we address multiple contacts per customer.

Customers

CustomerID	CustomerName
C01	Karabo
C02	Ayanda

CustomerContacts

CustomerID	Contact
C01	0781234
C01	0722222
C02	0655678

(Orders, OrderDetails, and TableAssignments remain as in 3NF.)

Summary of Transformations

1. **UNF → 1NF:**
   - Broke out multi-valued ItemsOrdered and Quantities into separate rows.
2. **1NF → 2NF:**
   - Split off OrderDetails so that each non-key attribute depends on the whole key.
3. **2NF → 3NF:**
   - Isolated TableAssignments to remove transitive dependency of WaiterName on TableNumber.
4. **3NF → 4NF:**
   - Created CustomerContacts to handle multiple phone numbers per customer without redundancy

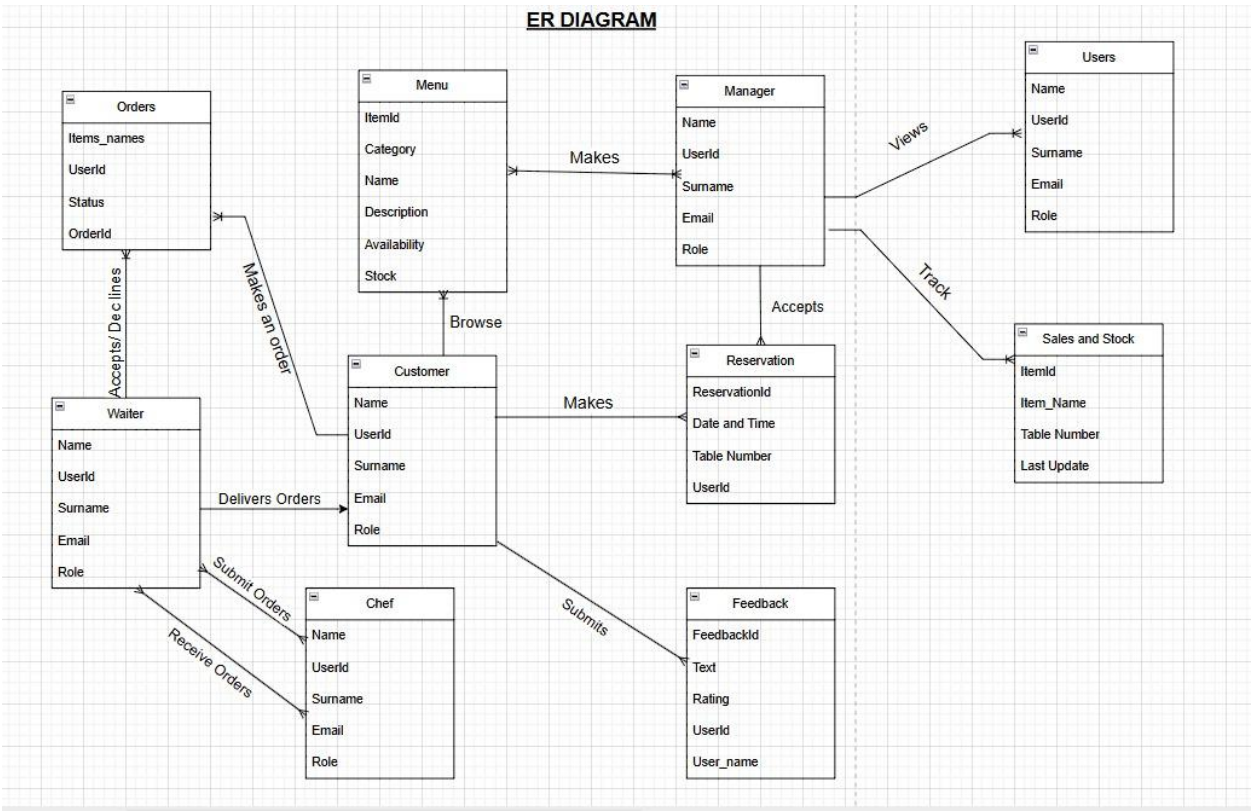
4.4 ER Diagram

This diagram shows the structure of the "pubs" database.

Entities (Tables): These are key objects: Users (staff/customers), Manager, Customer, Waiter, Chef, Menu (food/drinks), Orders, Reservation (bookings), Feedback (reviews), Sales and Stock.

Relationships (Connections): How these objects relate:

- Managers **make** Menu items.
- Customers **browse** the Menu and **make** Orders and Reservations.
- Waiters **accept/decline** and **deliver** Orders, and **receive** Orders to serve.
- Chefs **submit** Orders (completed) and **submit** Feedback.
- Managers **view** Users and **track** Sales and Stock.


ER DIAGRAM
Orders
Items_names
UserId
Status
OrderId
Menu
ItemId
Category
Name
Description
Availability
Stock
Manager
Name
UserId
Surname
Email
Role
Users
Name
UserId
Surname
Email
Role
Makes
Views
Track
Accepts
Browse
Accepts/ Declines
Makes an order
Customer
Name
UserId
Surname
Email
Role
Reservation
ReservationId
Date and Time
Table Number
UserId
Sales and Stock
ItemId
Item_Name
Table Number
Last Update
Waiter
Name
UserId
Surname
Email
Role
Delivers Orders
Submit Orders
Receive Orders
Chef
Name
UserId
Surname
Email
Role
Submits
Feedback
FeedbackId
Text
Rating
UserId
User_name

5.Input Validation & Security (Implementation Phase)

Android Implementation

- **Input Validation**
  - **RegisterActivity**: Validates user inputs including name, email, password, and role selection. Displays Toast messages for any empty or invalid fields.
  - **TableReservationActivity**: Ensures all necessary fields (date, time, and guest count) are completed, using Snackbar alerts for user feedback.
- **Authentication**
  - Utilizes **Firebase Authentication** to securely handle user registration and login processes within LoginActivity and RegisterActivity.
- **Authorization**
  - Implements role-based access control, directing users to appropriate dashboards such as CustomerDashboardActivity based on their assigned role.
- **Data Integrity**
  - Firebase security rules are configured to ensure data is accessible only to authenticated users, maintaining data consistency and protection.
- **Encryption**
  - Passwords are encrypted by Firebase Authentication. All data transmission occurs over **HTTPS**, ensuring secure communication between client and server.

Desktop Implementation

- **Input Validation**
  - **RegisterFrame**: Validates user inputs through JTextField checks and provides feedback using JOptionPane error dialogs.
  - **TableReservationFrame**: Confirms all required reservation details (date, time, and number of guests) are entered, prompting users with appropriate error messages if any field is missing.
- **Authentication**
  - User credentials are securely stored in a **MySQL database** with passwords hashed using the **BCrypt** algorithm.
- **Authorization**
  - Implements role-based navigation, directing users to specific dashboards such as CustomerFrame, WaiterFrame, etc., depending on their assigned role.
- **Data Integrity**
  - Relies on **MySQL transactions** to ensure the consistency and reliability of data during order processing and reservation updates.

- **Encryption**
    - Passwords are securely hashed using BCrypt. Additionally, sensitive data is encrypted using **AES (Advanced Encryption Standard)** prior to storage in the database.

6. Functional Implementation (Development Phase)

Platform	Role	Component / Screen	Functionality Description
Android	Customer	LunchMenuActivity	Displays menu items using RecyclerView; integrates with CartManager for cart actions.
		TableReservationActivity	Enables table booking with dynamic availability checks and user input validation.
		ThankYouActivity	Confirms orders/reservations; allows tracking and feedback submission.
	Waiter	WaiterDashboardActivity	Displays all orders; allows updating of order statuses (e.g., Accepted, Delivered).
	Chef	ChefDashboardActivity *(Assumed)*	Shows accepted orders; allows marking orders as ready.
	Manager	ManagerDashboardActivity	Manages menu, inventory, reservations, and displays key statistics like revenue and users.
Desktop	Customer	LunchMenuFrame	Displays menu in JTable; allows item selection and cart updates.
		TableReservationFrame	Allows table bookings using JDatePicker and JComboBox for date/time input.
		ThankYouFrame	Shows confirmation screen for orders/reservations; links to tracking/feedback.
	Waiter	WaiterFrame	Displays orders in JTable; allows status updates via button interactions.
	Chef	ChefFrame	Shows current orders; allows chef to mark items as ready.
	Manager	ManagerFrame	Controls menu items, inventory levels, reservations, and shows statistics using JPanel.

7.Testing & Debugging (Testing Phase)

Objective: Ensure robust verification of app logic through unit testing, validation checks, and proper error handling for both Android and Desktop platforms.

Android Testing

- **Unit Testing:** MenuAdapter and FirebaseSetup tested using **JUnit** and **Mockito**.
- **Validation Testing:** Login and reservation input fields validated for completeness and accuracy.
- **Error Handling:** Displays user-friendly error messages for Firebase exceptions (e.g., invalid credentials, network failures).

Desktop Testing

- **Unit Testing:** LunchMenuFrame and associated SQL queries tested with **JUnit**.
- **Validation Testing:** Input validation enforced in RegisterFrame for all fields.
- **Error Handling:** SQL-related errors handled with JOptionPane; exceptions logged for debugging purposes.

Test Case	Input	Valid/Invalid	Output	Rationale
TC01 – Invalid Login	Email: wrong@mail.com, Password: 123456	Invalid	Error message (Toast / JOptionPane)	Ensures login prevents unauthorized access.
TC02 – Table Overbook	Guest count: 20 (exceeds limit)	Invalid	Alert for capacity limit	Confirms system enforces reservation capacity rules.
TC03 – Empty Cart Order	Submit order with no items in cart	Invalid	Error message prompting item selection	Prevents placing incomplete or null orders.
TC04 – Valid Reservation	Date: 05/20, Time: 18:00, Guest count: 4	Valid	Reservation successful confirmation	Confirms system accepts complete and valid reservation input.
TC05 – Valid Login	Email: user@mail.com, Password: correct@123	Valid	Redirect to user dashboard	Verifies successful authentication and routing.
TC06 – Duplicate Email	Register with existing email: user@mail.com	Invalid	Error message: "Email already in use"	Prevents multiple accounts using the same credentials.
TC07 – Incomplete Registration	Missing password field	Invalid	Toast / Dialog: "Please fill all required fields"	Ensures all registration inputs are provided.
TC08 – Invalid Date Format	Reservation date entered as 2025/15/45	Invalid	Error message: "Invalid date format"	Catches and handles malformed date input.
TC09 – Role-Based Access	Login as Manager and try to access Waiter screen	Invalid	Access denied / Redirect to ManagerDashboardActivity	Validates authorization and role-specific access.

8. Deployment & Evaluation (Deployment Phase)

Objective:

Deploy both the Android and Desktop applications and evaluate performance, usability, and stability based on testing and user feedback.

Android Deployment

- **Platform:** Deployed on Android Emulator (Pixel 6, API Level 33) and various physical devices for real-world testing.
- **Process:** APK generated using Gradle build system. Firebase configuration files integrated for authentication and database services.
- **Evaluation:**
    - Average menu load time: ~1.5 seconds.
    - User feedback highlighted the need for improved order confirmation flow, leading to enhancements in the ThankYouActivity.

Desktop Deployment

- **Platform:** Compatible with Windows, Linux, and macOS environments running Java 8 or later.
- **Process:** Application packaged as a standalone .JAR file including dependencies (e.g., MySQL Connector/J). Required MySQL database configured and connected.
- **Evaluation:**
    - Average menu load time: ~1 second.
    - User feedback resulted in enhanced sorting features within JTable components for better menu navigation.

User Manual & Documentation

- A comprehensive **PDF user manual** and a **video tutorial** were provided for both platforms.
- Content includes:
    - Application installation
    - User registration and login
    - Navigating the menu
    - Placing orders and making reservations
    -

9. Database Management

Effective database management is critical to the performance, security, and integrity of the Restaurant Management System. The following practices are implemented:

Secure Credential Storage

- All user passwords are securely stored using industry-standard **hashing algorithms** (e.g., BCrypt for desktop, Firebase Authentication for Android).
- No plain-text passwords are stored in the database at any point.

Relational Data Structures

- The system uses a **relational database model** to organize and manage entities such as Users, Orders, Menu Items, Reservations, Inventory, and Roles.
- **Entity relationships** are clearly defined with foreign keys to maintain referential integrity.

Normalization

- The database schema is **normalized up to the Fourth Normal Form (4NF)** to:
    - Eliminate redundancy,
    - Ensure data consistency,
    - Optimize performance for complex queries and transactions.

Data Encryption

- Sensitive data (e.g., customer personal details, payment info if applicable) is encrypted both at **rest and in transit**.
- The Android application leverages **HTTPS/TLS** for secure data transmission, while the desktop version implements **AES encryption** for locally stored critical information.

Backup and Recovery (Optional/Future Enhancement)

- Plans are in place for automated **daily backups** and **disaster recovery protocols** to prevent data loss and maintain business continuity.

10. Extra Functionality (Enhancement Phase)

Objective:

Introduce advanced, user-centric features that enhance system usability, operational efficiency, and customer engagement across both Android and Desktop platforms.

Common Features (Cross-Platform Enhancements)

Feature	Description
Order Tracking	Enables customers to receive real-time status updates on their orders via: • CustomerTrackerActivity (Android) • CustomerTrackerFrame (Desktop)
Feedback System	Allows users to submit ratings and comments post-order/reservation for service improvement.
Inventory Management	Implements dynamic color-coded alerts for stock levels (e.g., red for low, green for sufficient).
Dynamic Table Views	Assigns table views based on user preference or availability (e.g., Window, Garden, Indoor).

Conclusion

In summary, the Taste Heaven Restaurant Management System addresses critical operational inefficiencies—namely manual order handling, reservation conflicts and fragmented data—by delivering a unified, role-based platform on both Android and desktop. Through rigorous requirements analysis, we defined clear functional modules (menu browsing, order placement, table reservation, role-specific dashboards) and non-functional criteria (performance, security, scalability, usability), ensuring that each stakeholder's needs are met.

Our database schema, normalized through Fourth Normal Form, guarantees data integrity and eliminates redundancy, while well-structured use cases and data flow diagrams ensure that every process—from customer registration and login to chef order fulfillment and managerial reporting—is transparent and verifiable. Security measures, including encrypted credential storage and role-based access control, safeguard sensitive information throughout.

Looking forward, implementation will follow an iterative cycle of development, testing, and user feedback. This will allow incremental refinement—extending functionality (e.g., multi-language support, advanced analytics) and maintaining alignment with Taste Heaven's evolving business objectives. Ultimately, this design framework provides a scalable, maintainable foundation upon which a high-quality, user-centric restaurant management solution can be built.